

Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Λειτουργικά Συστήματα Υπολογιστών

Γρίφος

Αλεξάνδρα Μυλωνά 03123004*
Ίωνας Τσακίρης 03123077†

22 Οκτωβρίου 2026

*el23004@mail.ntua.gr
†el23077@mail.ntua.gr

Σημείωση: Για τη διερεύνηση των εμποδίων χρησιμοποιούμε τα εργαλεία `ltrace` και `strace`, ώστε να παρακολουθούμε τις κλήσεις συναρτήσεων βιβλιοθηκών και τις κλήσεις συστήματος του εκτελέσιμου.

Η επίλυση αυτοματοποιείται μέσω προγράμματος σε C, το οποίο διαμορφώνει το κατάλληλο περιβάλλον εκτέλεσης και εκκινεί τον γρίφο σε διεργασία-παιδί με χρήση των fork() και execv().

Εμπόδιο 0

Διάγνωση

Χρησιμοποιώντας την `ltrace -SCf` βλέπουμε την κλήση `open(".hello_there", 0, 00) = -1`. Συμπεραίνουμε πως για την επίλυση απαιτείται η ύπαρξη του αρχείου `.hello_there` στον τρέχοντα κατάλογο εργασίας της διεργασίας.

```
[pid 3740] open(".hello_there", 0, 00 <unfinished ...>
```

Επίλυση

Δημιουργούμε το ζητούμενο αρχείο ώστε η κλήση να επιτυγχάνει, μέσω της `open()` με flag `O_CREAT`.

```
oslab11@oslab-vm:~/riddle-x86-64$ ./m

Challenge   0: 'Hello there'
Hint:          'Processes run system calls'.
SUCCESS
```

riddle: Welcome back challenger. You may pass.

Εμπόδιο 1

Διάγνωση

Όμοια, βλέπουμε πως γίνεται κλήση της `open(..., O_WRONLY, ...)` (δηλαδή ζητείται πρόσβαση εγγραφής στο αρχείο `.hello_there`.), την οποία θέλουμε να αποτρέψουμε.

```
[pid 3955] open(".hello_there", 1, 00 <unfinished ...>
```

Επίλυση

Αλλάζουμε την επίτρεψη του αρχείου σε `0444` (`r--r--r--`), δηλαδή σε read-only, χρησιμοποιώντας την `chmod()`.

```
Challenge   1: 'Gatekeeper'
Hint:          'Stand guard, let noone pass'.
SUCCESS
```

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 2

Διάγνωση

Από την κάτωθι έξοδο του `ltrace`, παρατηρούμε ότι ο γρίφος θέτει ένα `alarm(10)`, ένα `pause()` και handlers για τα `SIGCONT, SIGALRM`. Το `alarm(10)` προγραμματίζει την αποστολή του σήματος `SIGALRM` μετά από περίπου 10 δευτερόλεπτα, ενώ το `pause()` αναστέλλει την εκτέλεση της διεργασίας μέχρι να παραδοθεί κατάλληλο σήμα.

```
[pid 7545] signal(SIGALRM, 0x6207768bb350 <unfinished ...>
[pid 7545] SYS_rt_sigaction(14, 0x7fff6a5bc850, 0x7fff6a5bc8f0, 8)        = 0
[pid 7545] <... signal resumed> )                                         = 0
[pid 7545] signal(SIGCONT, 0x6207768bb350 <unfinished ...>
[pid 7545] SYS_rt_sigaction(18, 0x7fff6a5bc850, 0x7fff6a5bc8f0, 8)        = 0
[pid 7545] <... signal resumed> )                                         = 0
[pid 7545] alarm(10 <unfinished ...>
[pid 7545] SYS_alarm(10, 0x7fff6a5bc850, 0, 8)                            = 0
[pid 7545] <... alarm resumed> )                                          = 0
[pid 7545] pause(10, 0x7fff6a5bc850, 0, 0x7625aecee33b <unfinished ...>
[pid 7545] SYS_pause()                                                    = -514
[pid 7545] --- SIGALRM (Alarm clock) ---
[pid 7545] SYS_rt_sigreturn(14, 0x7fff6a5bbf30, 0x7fff6a5bbe00, 8)        = -4
[pid 7545] <... pause resumed> )                                          = 0xffffffff
```

Επίλυση

Εκτελούμε τον γρίφο ως παιδί ενός άλλου προγράμματος, το οποίο του στέλνει `SIGCONT` πρίν τη λήξη του χρονόμετρου, μέσω της `kill(p, SIGCONT)`. Όταν το σήμα παραδοθεί κατά την αναμονή στο `pause()`, ενεργοποιείται ο αντίστοιχος handler και η διεργασία μπορεί να συνεχίσει. Στην υλοποίησή μας χρησιμοποιείται η `sleep(1)` πριν από την αποστολή του σήματος, ώστε να δοθεί χρόνος στο παιδί να φτάσει στο `pause()`. Η καθυστέρηση αυτή αποδεικνύεται επαρκής στην πράξη, αλλά δεν εξασφαλίζει αυστηρό συγχρονισμό μεταξύ των διεργασιών.

```
Challenge   2: 'A time to kill'
Hint:          'Stuck in the dark, help me move on'.
sending SIGCONT signal to 12958
SUCCESS
```

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 3

Διάγνωση

Όμοια με το εμπόδιο 6 του αντίστοιχου γρίφου του εισαγωγικού μαθήματος, παρατηρούμε ότι ο γρίφος καλεί την `getenv("ANSWER")`, η οποία επιστρέφει `NULL`, υποδεικνύοντας ότι η μεταβλητή περιβάλλοντος `ANSWER` δεν έχει οριστεί.

```
getenv("ANSWER")                        = nil
```

Επίλυση

Στο πρόγραμμα C χρησιμοποιούμε την `setenv("ANSWER", "42", 1)` πριν κάνουμε `fork()`. Το παιδί κληρονομεί το περιβάλλον του γονέα και το διατηρεί κατά την `execv()`, επιτρέποντας στον γρίφο να διαβάσει την επιθυμητή τιμή.

```
Challenge   3: 'what is the answer to life the universe and everything?'
Hint:          'ltrace'.
SUCCESS
```

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 4

Διάγνωση

Ο γρίφος ανοίγει το αρχείο `magic_mirror` με `O_RDWR`, γράφει έναν χαρακτήρα και επιχειρεί να τον διαβάσει από τον ίδιο file descriptor. Σε ένα κανονικό αρχείο, η `write()` μετακινεί το file offset, με αποτέλεσμα η `read()` να επιστρέφει 0 (EOF).

```
openat(AT_FDCWD, "magic_mirror", O_RDWR) = 4
write(4, "I", 1)                        = 1
read(4, "", 1)                          = 0
write(2, "\33[31mI cannot see my reflection. FAIL\33[0m\n", 42I cannot see my reflection. FAIL
) = 42
```

Επίλυση

Δημιουργούμε ένα named FIFO pipe για τη διεργασία (βλ. `mkfifo(3)`) πριν από τη `fork()`. Σε αντίθεση με ένα κανονικό αρχείο, το FIFO μεταφέρει δεδομένα μέσω buffer του πυρήνα, χωρίς file offset, επιτρέποντας στον γρίφο να διαβάσει τον χαρακτήρα που μόλις έγραψε.

```
Challenge   4: 'First-in, First-out'
Hint:          'Mirror, mirror on the wall, who in this land is fairest of all?'.
SUCCESS
```

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 5

Διάγνωση

Γίνεται κλίση της `fcntl` με `fildes = 99`.

Επίλυση

Δημιουργούμε ένα νέο αρχείο με δικαιώματα ανάγνωσης και γραφής, και με χρήση του `dup2()` του αναθέτουμε ως file descriptor τον αριθμό 99.

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 6

Διάγνωση

Παρατηρούμε ότι ο γρίφος γεννά δύο διεργασίες, οι οποίες διαδοχικά γράφουν και διαβάζουν από τα file descriptors 33, 34, 53, 54, ..., χρησιμοποιώντας εξωτερικές διακοπές, ορισμένες με κλήση της `alarm()`, για να συγχρονιστούν.

[pid 23930] __fprintf_chk(0x7f3b9081a4a0, 1, "Hint:·····'%s'.\n", "help-us-play!"Hint:
'help-us-play!'.
) = 32
[pid 23930] getenv("I_NEED_TECH_HINTS") = nil
[pid 23930] fork() = 23931
[pid 23930] fork(<unfinished ...>
[pid 23931] <... fork resumed>) = 0
[pid 23931] getpid(<unfinished ...>
[pid 23930] <... fork resumed>) = 23932
[pid 23930] **wait**(0x7ffcb7efaa74 <unfinished ...>
[pid 23932] <... fork resumed>) = 0
[pid 23932] alarm(2 <unfinished ...>
[pid 23931] <... getpid resumed>) = 23931
[pid 23931] __printf_chk(1, "[%d]·PING!\n", 23931 <unfinished ...>
[pid 23932] <... alarm resumed>) = 0
[pid 23932] **read**(33 <no **return** ...>
error: maximum array length seems negative
, "admi", 4) = −1
[pid 23932] alarm(0) = 2
[23931] PING!
[pid 23932] **exit**(1 <unfinished ...>
[pid 23931] <... __printf_chk resumed>) = 14
[pid 23931] write(34, "", 4 <no **return** ...>
[pid 23932] +++ exited (status 1) +++
[pid 23930] −−− SIGCHLD (Child exited) −−−
[pid 23930] <... **wait** resumed>) = 23932
[pid 23930] **wait**(0x7ffcb7efaa74 <unfinished ...>
[pid 23931] <... write resumed>) = −1
[pid 23931] **exit**(1 <no **return** ...>
[pid 23931] +++ exited (status 1) +++
[pid 23930] −−− SIGCHLD (Child exited) −−−
[pid 23930] <... **wait** resumed>) = 23931

Επίλυση

Δημιουργούμε μια συστοιχία από pipes, ορίζοντας ως file descriptors – με χρήση της `dup2()` – τις τιμές των ακολουθιών $33 + 20n$ και $34 + 20n$ για τις άκρες ανάγνωσης και εγγραφής αντίστοιχα. Στη συνέχεια, οι δίαυλοι αυτοί κληρονομούνται από τον γρίφο και τα παιδιά του, επιτρέποντας τη συναμεταξύ τους επικοινωνία.

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 7

Διάγνωση

[pid 6489] __lxstat(1, ".hello_there", 0x7ffedf210b40 <unfinished ...>
[pid 6489] lstat@SYS(".hello_there", 0x7ffedf210b40) = 0
[pid 6489] <... __lxstat resumed>) = 0
[pid 6489] __lxstat(1, ".hey_there", 0x7ffedf210bd0 <unfinished ...>
[pid 6489] lstat@SYS(".hey_there", 0x7ffedf210bd0) = 0
[pid 6489] <... __lxstat resumed>) = 0

Επίλυση

Δημιουργούμε ένα hard link: .hey_there προς το αρχείο .hello_there.

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 8

Διάγνωση

[pid 8452] open("bf00", 0, 021 <unfinished ...>
[pid 8452] openat@SYS(AT_FDCWD, "bf00", 0, 00) = 4
[pid 8452] <... open resumed>) = 4
[pid 8452] lseek(4, 1073741824, SEEK_SET <unfinished ...>
[pid 8452] lseek@SYS(4, 1073741824, 0) = 1073741824
[pid 8452] <... lseek resumed>) = 1073741824
[pid 8452] **read**(4 <unfinished ...>
[pid 8452] read@SYS(4, "", 16) = 0
[pid 8452] <... **read** resumed> , "", 16) = 0
[pid 8452] __printf_chk(1, "%s", "X" <unfinished ...>
[pid 8452] fstat@SYS(1, 0x7fff263104a0) = 0
[pid 8452] <... __printf_chk resumed>) = 1
[pid 8452] fflush(0x7f8a7961a580 <unfinished ...>
[pid 8452] write@SYS(1, "X", 1X) = 1

[pid 8452] <... fflush resumed>) = 0
[pid 8452] close(4 <unfinished ...>
[pid 8452] close@SYS(4) = 0
[pid 8452] <... close resumed>) = 0
...
[pid 8452] write@SYS(1, "X", 1X) = 1
[pid 8452] <... fflush resumed>) = 0
[pid 8452] close(4 <unfinished ...>
[pid 8452] close@SYS(4) = 0
[pid 8452] <... close resumed>) = 0
[pid 8452] open("bf00", 0, 00 <unfinished ...>
[pid 8452] openat@SYS(AT_FDCWD, "bf00", 0, 00) = 4
[pid 8452] <... open resumed>) = 4
[pid 8452] lseek(4, 1073741824, SEEK_SET <unfinished ...>
[pid 8452] lseek@SYS(4, 1073741824, 0) = 1073741824
[pid 8452] <... lseek resumed>) = 1073741824
[pid 8452] **read**(4 <unfinished ...>
[pid 8452] read@SYS(4, "", 16) = 0
[pid 8452] <... **read** resumed> , "", 16) = 0
[pid 8452] __printf_chk(1, "%s", "X") = 1
[pid 8452] fflush(0x7f8a7961a580 <unfinished ...>
[pid 8452] write@SYS(1, "X", 1X) = 1
[pid 8452] <... fflush resumed>) = 0
[pid 8452] close(4 <unfinished ...>
[pid 8452] close@SYS(4) = 0
[pid 8452] <... close resumed>) = 0

Επίλυση

Αρκεί να δημιουργήσουμε μία συστοιχία αρχείων, τα οποία υποστηρίζουν εγγραφή και ανάγνωση, με μέγεθος 1073741824 (1.1GB). Στην προκειμένη περίπτωση, αξιοποιούμε τη δυνατότητα του συστήματος αρχείων ext4 να υποστηρίζει αραιά αρχεία, όπου τυχόν «τρύπες» μεγάλης έκτασης δε γεμίζονται με μηδενικά, όπως στο vfat. Έτσι, το πραγματικό μέγεθος των αρχείων είναι πολύ πιό μικρό από το απαιτούμενο.

Εναλλακτικά, με χρήση του FUSE θα μπορούσαμε να κατασκευάσουμε ένα σύστημα ψεύδο-αρχείων, τα οποία φαινομενικά έχουν το ζητούμενο μέγεθος και οι κλήσεις ανάγνωσης επιστρέφουν μια συμβολοσειρά της επιλογής μας.

`riddle:` `Welcome back challenger. You may pass.`

Εμπόδιο 9

Διάγνωση

Έξοδος `ltrace -SCf`:

[pid 14986] socket(2, 1, 0 <unfinished ...>

[pid 14986] socket@SYS(2, 1, 0, 0) = 4
[pid 14986] <... socket resumed>) = 4
[pid 14986] inet_aton("127.0.0.1", {0x100007f}) = 1
[pid 14986] fputs("I-am-trying-to-contact-you...\\n", 0x7fb1ae41a4a0 <unfinished ...>
[pid 14986] write@SYS(2, "I-am-trying-to-contact-you...\\n", 31I am trying to contact you...\n) = 31
[pid 14986] <... fputs resumed>) = 1
[pid 14986] connect(4, 0x7ffe7e4c6250, 16, 0x7fb1ae41b740 <unfinished ...>
[pid 14986] connect@SYS(4, 0x7ffe7e4c6250, 16, 0) = −111
[pid 14986] <... connect resumed>) = 0xffffffff
[pid 14986] fputs("...but-you-don't-seem-to-listen."..., 0x7fb1ae41a4a0 <unfinished ...>
[pid 14986] write@SYS(2, "...but-you-don't-seem-to-listen."..., 34...but you don't-seem-to-listen.\n)~~~~~~=-34
[pid-14986]-<...-fputs-resumed>-)~~~~~~~~~~~~~~~~~~~

unlink("secret_number") = 0
write(4, "The number I am thinking of right now is [uppercase matters]: \0\0\0\0\0\..."..., 4096) = 4096
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7f46b8703000
close(4) = 0
write(2, "What hex number am I thinking of right now? ", 44What hex number am I thinking of right now?) = 44
fstat(0, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
read(0BE3F5154
, "BE3F5154\n", 1024) = 9
write(2, "\33[32mSUCCESS\33[0m\n", 17SUCCESS

Επίλυση

Παρατηρούμε ότι ο γρίφος καταγράφει τον ζητούμενο αριθμό στο αρχείο `secret_number`, το οποίο αφ' ενός διαγράφει με κλίση της `unlink()`, αφ' ετέρου χαρτογραφεί στην εικονική του μνήμη με χρήση της `mmap()`. Καταφέρνουμε να διαβάσουμε τον αριθμό δημιουργώντας το `secret_number` εκ των προτέρων, μαζί με ένα hard link προς αυτό, έτσι ώστε να συνεχίσει να είναι διαθέσιμο ακόμη και μετά την κλήση της `unlink()`.

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 11

Διάγνωση

Όμοια με το Εμπόδιο 10, μόνο που γίνεται έλεγχος του reference count του αρχείου.

Επίλυση

Μη μπορώντας να χρησιμοποιήσουμε τη λύση του 10ου εμποδίου, αναζητούμε τον αριθμό στην εικονική μνήμη του γρίφου. Συγκεκριμένα, μεταβαίνουμε στον κατάλογο `/proc/<riddle_pid>/map_files` και βρίσκουμε τη χαρτογράφηση του αρχείου βάσει του ονόματος magic_mirror στην ανεπτυγμένη διεύθυνση των symlinks. Εναλλακτικά, θα μπορούσαμε να χρησιμοποιήσουμε τη διεύθυνση εικονικής μνήμης που επιστρέφει η κλήση της `mmap()`. Χάριν απλότητας, η λύση του εμποδίου 11 έχει υλοποιηθεί σε Bash και παρατίθεται στο αρχείο `src/C11.sh`.

```
riddle: Welcome back challenger.  You may pass.
```

Εμπόδιο 12

Διάγνωση

Ο γρίφος δημιουργεί ένα αρχείο της μορφής `/tmp/riddle-XXXXXX`, εξασφαλίζοντας την εκ νέου δημιουργία του με χρήση της μάσκας `O_EXCL`, και στη συνέχεια το χαρτογραφεί στη

μνήμη του. Το ζητούμενο είναι να αποκτήσουμε πρόσβαση στο αρχείο αυτό, να μεταβούμε σε ένα offset και να γράψουμε έναν τυχαίο χαρακτήρα που δίνεται από τον γρίφο.

```
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE,
    MAP_DROPPABLE|MAP_ANONYMOUS, −1, 0) = 0x7fc979be5000
mmap(NULL, 4096, PROT_READ|PROT_WRITE,
    MAP_PRIVATE|MAP_ANONYMOUS, −1, 0) = 0x7fc979be4000
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
getrandom("\x31\x86\x87\xf5\x67\x84\xa1\xbf\x09\x83\x1a\xfc\xf4\x32\x17\x4e\xa2...",
    32, 0) = 32
openat(AT_FDCWD, "/tmp/riddle−QU2uSt", O_RDWR|O_CREAT|O_EXCL, 0600)
    = 4
ftruncate(4, 4096) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
    0x7fc979be3000
write(2, "I-want-to-find-the-char-'X'-at-0"..., 46I want to find the char 'X' at
    0x7fc979be306f
) = 46
```

Επίλυση

Η δοθείσα διεύθυνση προκύπτει από τη θέση του αρχείου στην εικονική μνήμη της διεργασίας, συν μια μετατόπιση. Δεδομένου ότι το μέγεθος ενός μπλοκ μνήμης είναι 4MB, λαμβάνουμε το offset ως τη τελευταία διεύθυνση $\mod 0x1000$. Ακόμη, επειδή το ήμισυ του ονόματος του αρχείου παράγεται τυχαία, λαμβάνουμε το ορθό όνομα διατρέχοντας τα symlinks του `/proc/<riddle_pid>/map_files`, όμοια με το 11ο εμπόδιο, μόνο που εδώ εκτελούμε την εντολή Bash με χρήση της `system()`.

Σημειώνουμε ότι, αγνοώντας την επίκλιση προς την απλότητα της λύσης, λαμβάνουμε τη διεύθυνση και τον χαρακτήρα από τον γρίφο ανακατευθύνοντας την έξοδό του σε ένα pipe προς τη γονεϊκή μας διεργασία, όπου την αναλύουμε. Απόρροια της επιλογης αυτής είναι η απόκρυψη του μηνύματος επιτυχίας από τον χρήστη.

Εμπόδιο 13

Διάγνωση

Δημιουργείται ένα αρχείο `.hello_there`, επεκτείνεται σε `0x8000`B, χαρτογραφείται στη μνήμη και στη συνέχεια περικόπτεται σε `0x4000`B με διαδοχικές κλήσεις της `ftruncate`. Στη συνέχεια, ο γρίφος περιμένει έως ότου του δοθεί κάποια είσοδος, όποτε και αποπειράται να προσπελάσει να περικομμένη μνήμη.

```
[pid 31180] open(".hello_there", 0x42, 0600 <unfinished ...>
[pid 31180] openat@SYS(AT_FDCWD, ".hello_there", 0x42, 0600) = 4
[pid 31180] <... open resumed> ) = 4
[pid 31180] ftruncate(4, 32768 <unfinished ...>
[pid 31180] ftruncate@SYS(4, 32768) = 0
[pid 31180] <... ftruncate resumed> ) = 0
```

[pid 31180] mmap(nil, 32768, 0b11, 0x1, 4, 0 <unfinished ...>
[pid 31180] mmap@SYS(nil, 32768, 3, 1, 4, 0) = 0x7f61115f7000
[pid 31180] <... mmap resumed>) = 0x7f61115f7000
[pid 31180] ftruncate(4, 16384 <unfinished ...>
[pid 31180] ftruncate@SYS(4, 16384) = 0
[pid 31180] <... ftruncate resumed>) = 0

Επίλυση

Όσο περιμένει ο γρίφος, καλούμε την `ftruncate` και επανεπεκτείνουμε το αρχείο στα `0x8000`B, οπότε η μετέπειτα προσπέλαση επιτυγχάνει.

Εμπόδιο 14

Διάγνωση

Βάσει της υπόδειξης του γρίφου, απαιτεί να έχει PID = 32767.

Επίλυση

Υπάρχουν δύο πιθανές λύσεις: η τροποποίηση του `/proc/sys/kernel/ns_last_pid`, και η δημιουργία ενός νέου `pid_namespace(7)`. Εκ των δύο, καταφέραμε να υλοποιήσουμε τη δεύτερη, παρ΄ όλ΄ αυτά παρατίθενται και οι δύο κάτωθι για λόγους πληρότητας.

Η πρώτη μέθοδος προϋποθέτει την ενεργοποίηση της σημαίας `CONFIG_CHECKPOINT_RESTORE` στον πυρήνα, έτσι ώστε να καταγράφεται το τελευταίο PID που χρησιμοποιήθηκε στο αρχείο `ns_last_pid`. Τότε, αρκεί να ανοίξουμε το αρχείο αυτό - κλειδώνοντάς το κατάλληλα - και να γράψουμε το ζητούμενο PID - 1, με αποτέλεσμα η διεργασία που προκύπτει από τον μετέπειτα `fork` να έχει το επιθυμητό PID.

Εναλλακτικά, με χρήση της `clone3(2)` μπορούμε να δημιουργήσουμε ένα ανεξάρτητο `pid_namespace` θέτοντας τη σημαία `CLONE_NEWPID` κατά τη δημιουργία της πρώτης διεργασίας του namespace. Η τελευταία έχει όμοιο ρόλο με την init, και μπορεί να ορίσει το PID των παιδιών της - παροτρύνοντας την επιλογή από μια λίστα πιθανών PID. Οπότε, καλούμε για άλλη μια φορά την `clone3`, δίδοντας ως μοναδική επιλογή τον ζητούμενο αριθμό.

Εμπόδιο 1

Διάγνωση

Επίλυση